Canonical Capability Allow-list — Implementation Summary

Pilot Execution Sprint. Restricts assignment pickers to the 12 canonical capability codes while preserving the catalogue page and any existing legacy assignment rows. No schema migration. No mobile changes.

1. Origin of the legacy capabilities

Source	Verdict
`Capability` table	**Yes — they live here.** Inserted at migration time, not at runtime.
Seed data (`prisma/seed.ts`)	**No.** Seed only inserts the 12 canonical codes.
Prisma migration `20260521113000_sprint_g1_operational_configuration_foundation/migration.sql`	**Yes — root cause.** Lines 229–249 insert 19 `Capability` rows under `ON CONFLICT (code) DO NOTHING`, of which seven are legacy: `SURVEY` (line 239), `REPAIR` (241), `CIVIL` (242), `DISTRIBUTION` (243), `THIRTY_THREE_KV` (244), `EMERGENCY_RESPONSE` (245), `OTHER` (248). The earlier migration `20260519024954` only defined the `OrganizationCapabilityType` and `OperationalDomain` enums — unrelated to the Capability table contents.
API response	Returned from `/enterprise/options.capabilities` (`enterprise.service.ts:getOptions`), which is what the assignment pickers consume.
Other	None.

These rows were applied to every environment that has run the Sprint G1 migration, so legacy codes appear in production. Removing them at the DB level would violate the “do not delete legacy records” requirement.

2. Allow-list — design

Two layers, both gated by the same allow-list of 12 canonical codes (Workspace × 2 + Governance × 2 + Asset Domains × 8):

Server side — primary filter

`/enterprise/options.capabilities` now returns only canonical + active rows:

```
this.prisma.capability.findMany({
  where: {
    isActive: true,
    code: { in: Array.from(CANONICAL_CAPABILITY_CODES) },
  },
  ...
})
```

The catalogue endpoint `/enterprise/capabilities` (used by the `/capabilities` admin page) is **unchanged** — it continues to return every row so legacy capabilities remain visible, editable, and deactivatable in the catalogue UI.

Client side — defense-in-depth

Both pickers filter `options.capabilities` through `isAssignableCapability` before calling `groupCapabilities`. If a future build of the server forgets the filter, the picker still keeps legacy codes out.

3. Preservation of existing data

- No DELETE statements anywhere. Legacy `Capability` rows remain in the table.
- No mutation of existing `*Capability` assignment rows.
- Existing entities that already have legacy assignments: their form state loader (`readUserCapabilityIds`, `readCapabilityIds`) still reads all assignments including legacy IDs. The toggle handler only adds or removes the specific ID it owns — legacy IDs in `values` are never touched because no checkbox exists for them. On save, the form sends back the original legacy IDs alongside any new canonical selections. Legacy assignments persist silently.
- New entities start with empty `capabilityIds`; the picker only allows canonical selection.

4. Files changed

File	Status	Change
`apps/api/src/common/canonical-capabilities.ts`	**new**	Exports `CANONICAL_CAPABILITY_CODES` set + `isCanonicalCapabilityCode()` predicate.
`apps/api/src/enterprise/enterprise.service.ts`	edited	Imported the constant; added `where: { isActive: true, code: { in: ... } }` to the `prisma.capability.findMany` call inside `getOptions()`. `listCapabilities` (catalogue endpoint) unchanged.
`apps/admin-web/src/lib/capability-groups.ts`	edited	Added explicit `ASSET_DOMAIN_CODES` set, derived `CANONICAL_CAPABILITY_CODES` from the union of the three group sets, exported `isCanonicalCapabilityCode()` and `isAssignableCapability()` (codes-canonical AND `isActive !== false`).
`apps/admin-web/src/components/enterprise-list-client.tsx`	edited	`CapabilityPicker` now calls `options.capabilities.filter(isAssignableCapability)` before grouping. Returns `null` if no assignable capabilities remain. Used by Organizations / Branches / MAINHEADs / Teams modals.
`apps/admin-web/src/components/users-client.tsx`	edited	`UserCapabilityPicker` applies the same filter. Used by Create User and Edit User.

No schema changes. No data migrations. No mobile changes (mobile does not call `/enterprise/options`).

5. Validation results

Step	Command	Result
API build	`pnpm --filter @ascure/api exec tsc -p tsconfig.build.json`	**PASS** — exit 0, no diagnostics.
Admin typecheck	`pnpm --filter @ascure/admin-web typecheck`	**PASS** — exit 0, no diagnostics.
Admin build	`pnpm --filter @ascure/admin-web build`	**PASS** — `next build` compiled successfully, all 21 routes generated.

All three ran clean on first attempt.

6. Behavior matrix after deploy

Scenario	Picker shows	Save payload	Stored assignments
Create new Organization → tick SAVR + INSPECTION	SAVR, INSPECTION (and other canonical)	`[SAVR_id, INSPECTION_id]`	SAVR, INSPECTION
Edit Organization that has INSPECTION + CIVIL → tick MAINTENANCE	INSPECTION ticked; CIVIL not shown; MAINTENANCE available	`[INSPECTION_id, CIVIL_id, MAINTENANCE_id]`	INSPECTION, CIVIL, MAINTENANCE (CIVIL preserved)
Edit Team that has only OTHER + REPAIR → save with no change	Empty picker (no canonical assigned)	`[OTHER_id, REPAIR_id]`	OTHER, REPAIR (preserved)
Catalogue page `/capabilities`	All 19 capability rows	n/a	n/a
Set legacy capability `isActive=false` via catalogue page	Picker already excludes (active filter + canonical filter)	n/a	n/a

7. Rollout notes

- **Deploy API first, admin web second.** Filtering the API response is the source of truth; the admin picker filter is belt-and-braces.
- **No mobile coordination needed.** Mobile does not consume `/enterprise/options` for capabilities.
- **No DB migration.** The seven legacy rows remain in `Capability` with their existing `isActive=true` status; the application-level filter excludes them. If product later decides to formally retire them, deactivating via the catalogue page (`isActive=false`) is sufficient and reversible. A real `DELETE` is still not recommended because of any historical assignment FKs.
- **Catalogue page visibility unchanged.** Per the brief, all 19 capabilities remain visible and editable from `/capabilities`. If a stricter posture is desired later, the catalogue endpoint can also be filtered or a “Legacy” section can be introduced.
- **Rollback.** Revert the three application files. No data side-effects. Filter helpers and constants remain harmless if reverted partially.